

Day 22 – Argo CD GitOps for GenAI Platforms

1. What you should learn today

By the end of Day 22, you should be able to explain and design:

- GitOps-based delivery for an EKS-hosted RAG platform.
- The responsibility boundary between CI and Argo CD.
- Argo CD reconciliation, drift detection, self-healing, pruning, hooks, waves, projects, and sync windows.
- Git-based promotion of an immutable application and AI-artifact release bundle.
- Secure secrets, RBAC, policy enforcement, observability, rollback, and disaster recovery.
- When to use Application, AppProject, ApplicationSet, Argo Rollouts, Image Updater, and app-of-apps.

The current stable Argo CD documentation continues to use:

```
apiVersion: argoproj.io/v1alpha1
```

for Application, AppProject, and ApplicationSet. Current External Secrets examples use:

```
apiVersion: external-secrets.io/v1
```

The examples below follow those APIs and avoid the removed PodSecurityPolicy resource. (argo-cd.readthedocs.io)

2. GitOps fundamentals

2.1 Declarative desired state

In imperative deployment, an operator says:

| Run these commands to change the cluster.

In declarative deployment, Git says:

| This is what the cluster should look like.

For example:

```
image:
  repository: 111122223333.dkr.ecr.ap-south-1.amazonaws.com/rag-api
  digest: sha256:83a6c0...
```

This is not a deployment command. It is the desired deployment state.

Argo CD repeatedly asks:

1. What does Git declare?
 2. What currently exists in Kubernetes?
 3. Are they different?
 4. Should I report or correct the difference?
-

2.2 Git as the source of truth

For GitOps-managed resources:

- Git is the approved desired state.
- Kubernetes is the currently observed live state.
- Argo CD reconciles live state toward Git.

This does **not** mean Git contains everything.

Git should contain:

- Kubernetes manifests.
- Helm charts and environment values.
- Image digests.
- Model and prompt identifiers.
- Vector-index versions.
- Secret references.
- Promotion evidence references.
- Argo CD configuration.

Git should not contain:

- Provider API keys.
- Database passwords.
- Private certificates.
- Raw production data.
- Unencrypted credentials.

2.3 Pull-based continuous delivery

In a traditional push deployment:

```
CI system —kubectl/helm access—> Kubernetes API
```

The CI system must hold cluster credentials and execute deployments.

In pull-based GitOps:

```
CI or release process —commit/PR—> Git
                                   |
                                   ▼
                               Argo CD
                                   |
                               Kubernetes API
```

Argo CD runs inside, or close to, the target Kubernetes environment and pulls the approved desired state.

A major security benefit is that the build system does not need unrestricted production-cluster credentials. Automated sync is specifically designed so that a pipeline can update Git rather than directly call the Argo CD API or Kubernetes API. (argo-cd.readthedocs.io)

2.4 Continuous reconciliation

Argo CD is not a one-time Helm execution.

The application controller continuously compares:

Rendered Git manifests
versus
Kubernetes live resources

If a Deployment is manually changed from three replicas to ten, Argo CD detects that live state differs from Git.

Depending on policy, it can:

- Report OutOfSync.
- Correct the change automatically.
- Wait for an authorized manual sync.
- Ignore a narrowly configured difference.

The application controller obtains desired manifests from the repository server and obtains actual state from the Kubernetes API. (argo-cd.readthedocs.io)

2.5 Drift detection and self-healing

Drift means the live Kubernetes object no longer matches the desired Git representation.

Examples:

- Someone manually edits a Deployment.
- Another controller changes a field.
- A ConfigMap is deleted.
- A Service selector is changed.
- A security context is removed.
- A production image is manually replaced.

With `selfHeal: true`, Argo CD can reapply the desired state when live resources drift.

Use self-healing for fields that should exclusively be controlled through Git. Do not enable it blindly where another legitimate controller owns the field.

2.6 Auditability

A properly protected deployment repository tells you:

- What changed.
- Who proposed the change.
- Who approved it.
- Which release entered each environment.
- Which image, model, prompt, and index were deployed.
- Why the promotion occurred.
- What evaluation evidence supported it.
- Which Git commit represented production.

This is much stronger than an unaudited sequence of:

```
kubectl set image ...  
helm upgrade ...  
kubectl edit ...
```

2.7 GitOps compared with push-based deployment

Concern	Push-based CD	Argo CD GitOps
Deployment trigger	Pipeline runs deployment command	Git change is detected
Cluster credentials	Usually held by CI	Held by Argo CD
Desired state	Pipeline parameters or release state	Git
Drift detection	Usually separate or absent	Continuous
Manual cluster edits	May remain indefinitely	Detected and optionally healed
Audit trail	Pipeline logs	Git history, PRs and Argo history
Rollback	Re-run an older deployment	Prefer Git revert
Network direction	CI connects to cluster	Argo CD pulls Git and accesses cluster

3. CI and GitOps CD responsibility boundary

3.1 CI responsibility

Your existing CI process performs:

```
Source commit  
↓  
Tests and security scans  
↓  
RAG evaluation and quality gates  
↓  
Build immutable image  
↓  
Sign and publish image to ECR  
↓  
Publish evaluation evidence
```

Example output:

```
Application image:  
111122223333.dkr.ecr.ap-south-1.amazonaws.com/rag-api  
@sha256:83a6c02a...  
  
Evaluation report:  
s3://genai-release-evidence/rag-api/2026-07-30.3/report.json  
  
Prompt bundle:  
prompt-bundle-2026-07-29.2  
sha256:53c1...
```

Vector compatibility contract:
rag-contract-v7

3.2 Controlled deployment-state update

After CI succeeds, a bot or release engineer opens a pull request against the deployment repository:

```
image:
- digest: sha256:5e9d...
+ digest: sha256:83a6c02a...

release:
- id: 2026-07-28.6
+ id: 2026-07-30.3
```

The PR contains or references the evidence produced by CI.

3.3 Argo CD responsibility

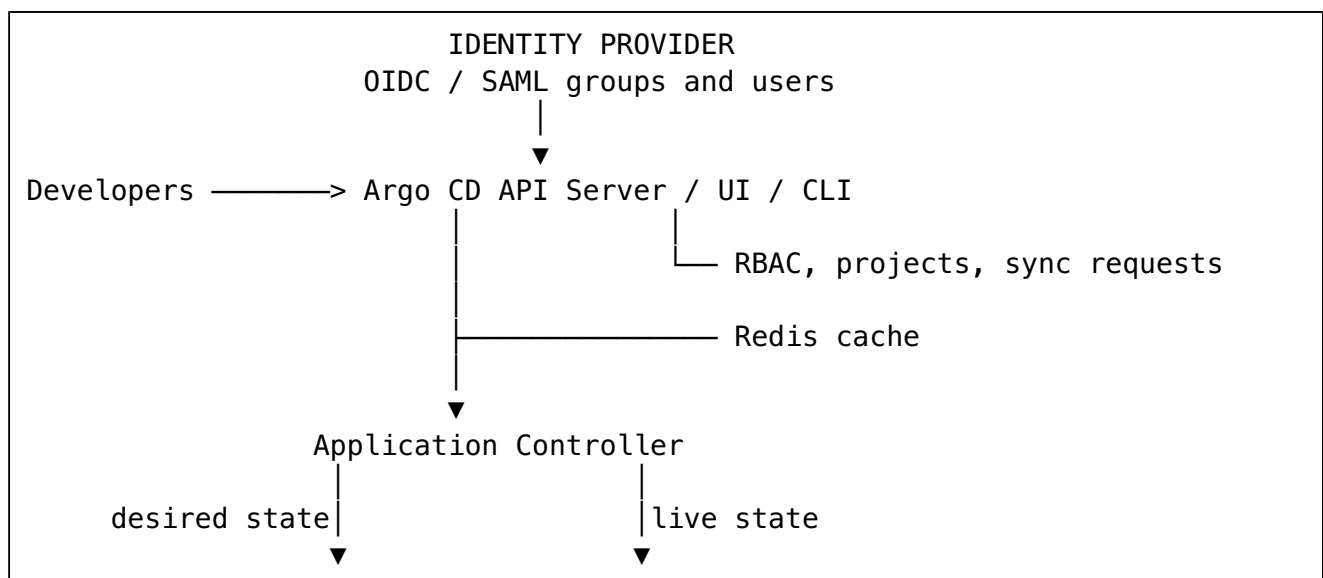
After the change is approved and merged:

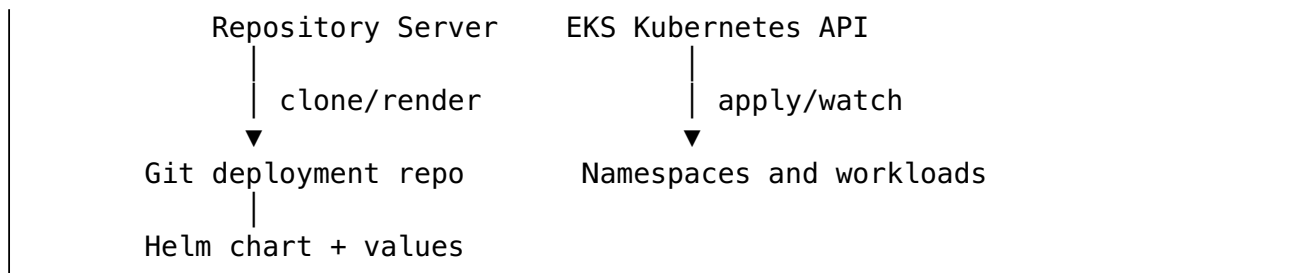
1. Repository server fetches Git.
2. Helm renders the chart with the updated values.
3. Application controller compares rendered desired state with live EKS state.
4. Argo CD marks the application OutOfSync.
5. Automatic or manual sync applies the change.
6. Kubernetes controllers perform the actual rollout.
7. Argo CD tracks synchronization and health.

Argo CD does not compile code, run unit tests, build images, or replace the CI system.

4. Architecture and request flow

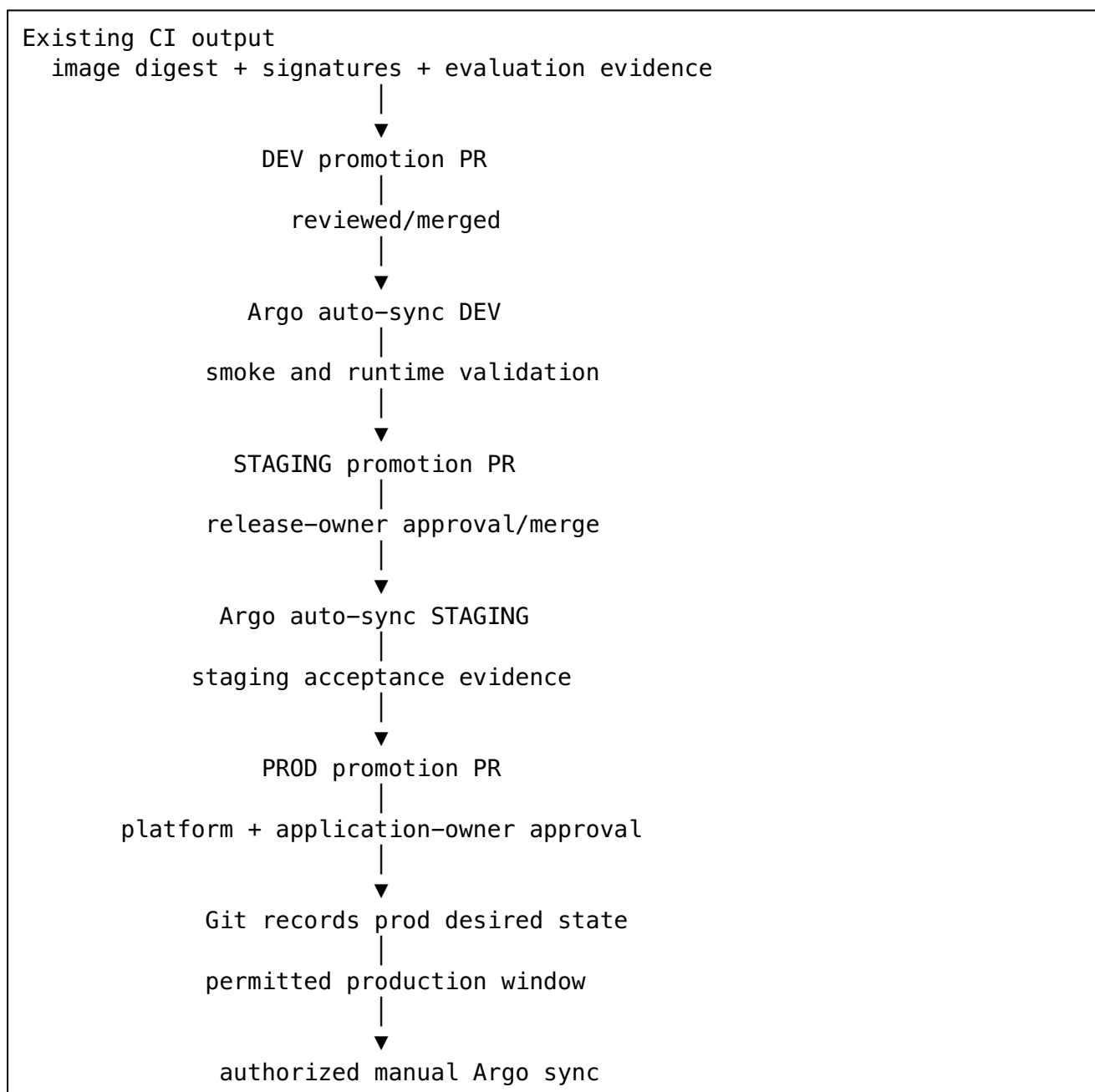
4.1 Architecture diagram





The API server exposes the REST/gRPC API consumed by the UI and CLI. The repository server clones repositories and generates manifests. The application controller continuously compares desired state with live state. Redis is primarily a cache used to reduce load on Git and Kubernetes, rather than the source of deployment truth. (argo-cd.readthedocs.io)

4.2 Promotion-flow diagram



5. Argo CD components

5.1 API server and UI

The Argo CD API server:

- Exposes REST and gRPC APIs.
- Serves the web UI.
- Authenticates users.
- Enforces Argo CD RBAC.
- Accepts application queries and sync requests.
- Integrates with an external identity provider.

It should not be treated as a public anonymous deployment endpoint.

In production:

- Expose it through a TLS-terminated ingress or load balancer.
- Prefer private/internal access where practical.
- Integrate with corporate SSO.
- Disable the built-in admin account after bootstrap.
- Restrict sync permission separately from read permission.

Argo CD has one built-in admin user and relies on an identity provider and RBAC for normal multi-user operation. OIDC groups can be mapped to Argo roles. (argo-cd.readthedocs.io)

5.2 Repository server

The repository server:

- Fetches Git repositories.
- Resolves revisions.
- Runs Helm, Kustomize or other manifest-generation tools.
- Returns generated Kubernetes manifests.
- Caches repository and rendering results.

It does not apply manifests to Kubernetes. The repository server is intentionally separated from Kubernetes privileges, reducing the impact of manifest-generation compromise. (argo-cd.readthedocs.io)

Potential production problems include:

- Repository-server memory exhaustion.
 - Temporary-disk exhaustion.
 - Slow Helm rendering.
 - Excessive monorepo invalidation.
 - Git authentication failure.
 - Rendering timeout.
 - Unpinned remote Helm dependencies.
-

5.3 Application controller

The application controller is the reconciliation engine.

For every Application, it:

1. Requests desired manifests from the repository server.
 2. Queries Kubernetes live resources.
 3. Calculates differences.
 4. Sets synchronization and health status.
 5. Executes authorized sync operations.
 6. Monitors resources until their health can be determined.
-

5.4 Redis

Redis supports caching and coordination.

It improves performance for:

- Application state.
- Repository data.
- Manifest-generation results.
- Controller and API-server communication.

Redis is not where your authoritative desired state should live. Git and Kubernetes resources remain the recoverable system state.

5.5 Identity-provider integration

A production setup normally uses:

```
Corporate identity provider
      ↓
OIDC/SAML integration
      ↓
Argo CD group claims
      ↓
Argo CD RBAC roles
```

Example separation:

- Developers: read all non-production applications and sync development.
 - QA/release team: sync staging.
 - Production release managers: sync production.
 - Platform administrators: manage projects, repositories and clusters.
 - Auditors: read-only access.
-

6. Installation and access on EKS

6.1 Bootstrap approach

For initial experimentation, the official installation manifest can be applied directly:

```
kubectl create namespace argocd

kubectl apply \
```

```
-n argocd \  
--server-side \  
--force-conflicts \  
-f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/  
install.yaml
```

The stable getting-started documentation currently shows server-side installation and CLI port-forwarding as supported access approaches. (argo-cd.readthedocs.io)

For production, improve this by:

- Pinning the Argo CD version rather than permanently following a floating URL.
- Managing Argo CD installation through a dedicated platform bootstrap repository.
- Using the HA installation or a maintained, pinned Helm release.
- Applying resource requests, PodDisruptionBudgets and topology constraints.
- Configuring TLS, SSO and RBAC before broad access.
- Backing up configuration before upgrades.

6.2 Local access

```
kubectl port-forward service/argocd-server \  
-n argocd \  
8080:443
```

Then:

```
argocd login localhost:8080  
argocd account get-user-info  
argocd version
```

For local CLI access without exposing the API server:

```
export ARGOCD_OPTS="--port-forward-namespace argocd"
```

6.3 Production exposure

Recommended EKS pattern:

```
Corporate network/VPN  
    ↓  
Internal ALB or ingress  
    ↓ TLS  
argocd-server  
    ↓  
Corporate OIDC provider
```

Avoid:

- Public unauthenticated UI exposure.
- Permanent use of the bootstrap admin password.
- `--insecure` as a production default.

- Broad ingress access from the internet.
 - Shared human API tokens.
 - Long-lived Git personal-access tokens.
-

6.4 Repository registration

Example CLI registration:

```
argocd repo add \
  git@github.com:example-org/genai-platform-deployments.git \
  --ssh-private-key-path ./argocd-deploy-key
```

Prefer:

- GitHub App credentials or dedicated deploy keys.
 - Read-only access for Argo CD.
 - Repository credentials scoped to the required organization or repository.
 - Secret rotation.
 - TLS host-key verification.
 - No developer personal token.
-

6.5 Cluster registration

For an Argo CD instance managing another cluster:

```
argocd cluster add genai-prod-eks
```

Before running this, inspect the generated permissions. Do not automatically grant unrestricted cluster-admin access.

For strong isolation, consider:

- One Argo CD installation per production cluster.
 - Separate Argo CD instances per high-trust boundary.
 - Namespace-scoped installation for constrained use cases.
 - Separate projects and destination credentials.
-

7. Repository design

7.1 Application repository versus deployment repository

Application repository

Contains:

```
FastAPI source
RAG orchestration code
Tests
Dockerfile
Evaluation definitions
Application-level Helm chart, optionally
```

Deployment/configuration repository

Contains:

```
Environment values
Approved image digests
Release metadata
Argo CD Applications
AppProjects
ApplicationSets
Policy configurations
Secret references
```

Separating these repositories gives stronger separation of duties:

- Developers create application artifacts.
- Release owners approve deployment state.
- Argo CD reads only deployment-approved configuration.

A monorepo remains acceptable for a smaller organization, but production promotion directories should still have protected ownership rules.

7.2 Complete example repository tree

```
genai-platform-deployments/
├── README.md
├── CODEOWNERS
├── charts/
│   └── rag-platform/
│       ├── Chart.yaml
│       ├── values.yaml
│       └── templates/
│           ├── serviceaccount.yaml
│           ├── externalsecret.yaml
│           ├── configmap.yaml
│           ├── migration-job.yaml
│           ├── api-deployment.yaml
│           ├── api-service.yaml
│           ├── ingestion-deployment.yaml
│           ├── evaluation-job.yaml
│           ├── networkpolicy.yaml
│           └── pdb.yaml
│       └── tests/
├── environments/
│   ├── dev/
│   │   ├── values.yaml
│   │   └── release-values.yaml
│   ├── staging/
│   │   ├── values.yaml
│   │   └── release-values.yaml
│   └── prod/
│       ├── values.yaml
│       └── release-values.yaml
└── argocd/
```

```

├── projects/
│   ├── genai-platform.yaml
│   ├── applications/
│   │   ├── rag-dev.yaml
│   │   └── applicationsets/
│   │       └── rag-environments.yaml
├── policies/
│   ├── approved-registries/
│   ├── require-image-digest/
│   ├── verify-image-signature/
│   └── require-security-context/
├── release-evidence/
│   └── 2026-07-30.3.yaml
├── bootstrap/
│   ├── root-application.yaml
│   └── namespaces/
│       ├── rag-dev.yaml
│       ├── rag-staging.yaml
│       └── rag-prod.yaml

```

The environment `values.yaml` files contain environment differences. The `release-values.yaml` files contain the release tuple being promoted.

This avoids copying complete Kubernetes manifests across environments.

8. Core resource: restricted AppProject

An AppProject is a security boundary. It limits:

- Allowed source repositories.
- Allowed destination clusters.
- Allowed namespaces.
- Allowed Kubernetes resource kinds.
- Project-specific RBAC roles.
- Sync windows.
- Orphan-resource behavior.

The official project specification supports source, destination and resource restrictions as well as sync windows. (argo-cd.readthedocs.io)

```

apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: genai-platform
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  description: Restricted project for the GenAI RAG platform

  sourceRepos:
    - git@github.com:example-org/genai-platform-deployments.git

```

```

destinations:
  - server: https://kubernetes.default.svc
    namespace: rag-dev
  - server: https://kubernetes.default.svc
    namespace: rag-staging
  - server: https://kubernetes.default.svc
    namespace: rag-prod

# No cluster-scoped resources are allowed from this project.
# Namespaces and platform CRDs are created by the platform bootstrap
project.
clusterResourceWhitelist: []

namespaceResourceWhitelist:
  - group: ""
    kind: ConfigMap
  - group: ""
    kind: Secret
  - group: ""
    kind: Service
  - group: ""
    kind: ServiceAccount
  - group: ""
    kind: ResourceQuota
  - group: apps
    kind: Deployment
  - group: apps
    kind: StatefulSet
  - group: batch
    kind: Job
  - group: batch
    kind: CronJob
  - group: autoscaling
    kind: HorizontalPodAutoscaler
  - group: policy
    kind: PodDisruptionBudget
  - group: networking.k8s.io
    kind: NetworkPolicy
  - group: external-secrets.io
    kind: ExternalSecret
  - group: argoproj.io
    kind: Rollout
  - group: argoproj.io
    kind: AnalysisTemplate

orphanedResources:
  warn: true

roles:
  - name: nonprod-deployer
    description: Sync development and staging only
    groups:
      - genai-nonprod-release
    policies:

```

```

- >-
  p, proj:genai-platform:nonprod-deployer,
  applications, get, genai-platform/*, allow
- >-
  p, proj:genai-platform:nonprod-deployer,
  applications, sync, genai-platform/rag-dev, allow
- >-
  p, proj:genai-platform:nonprod-deployer,
  applications, sync, genai-platform/rag-staging, allow

- name: production-deployer
  description: Approved production sync role
  groups:
    - genai-prod-release
  policies:
    - >-
      p, proj:genai-platform:production-deployer,
      applications, get, genai-platform/rag-prod, allow
    - >-
      p, proj:genai-platform:production-deployer,
      applications, sync, genai-platform/rag-prod, allow

syncWindows:
- kind: allow
  schedule: "0 9 * * 1-5"
  duration: 8h
  applications:
    - rag-prod
  namespaces:
    - rag-prod
  clusters:
    - https://kubernetes.default.svc
  manualSync: true

```

Important security points

1. The application project cannot create CRDs, namespaces, cluster roles or admission policies.
2. Infrastructure resources belong to a separate platform project.
3. Production is restricted by namespace, role and sync window.
4. Project permissions complement Git branch protection; they do not replace it.
5. The project does not permit arbitrary Git repositories.

9. Complete Argo CD Application

This development application uses:

- One approved repository.
- One Helm chart.
- Environment and release values.
- Automatic synchronization.
- Pruning and self-healing.
- Retry handling.

- Safe prune ordering.
- Shared-resource protection.

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: rag-dev
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
  labels:
    platform.example.com/team: genai
    platform.example.com/environment: dev
spec:
  project: genai-platform

  source:
    repoURL: git@github.com:example-org/genai-platform-deployments.git
    targetRevision: main
    path: charts/rag-platform
    helm:
      releaseName: rag-platform
      valueFiles:
        - ../../environments/dev/values.yaml
        - ../../environments/dev/release-values.yaml

  destination:
    server: https://kubernetes.default.svc
    namespace: rag-dev

  syncPolicy:
    automated:
      enabled: true
      prune: true
      selfHeal: true
      allowEmpty: false

  retry:
    limit: 5
    backoff:
      duration: 10s
      factor: 2
      maxDuration: 3m

  syncOptions:
    - PruneLast=true
    - ApplyOutOfSyncOnly=true
    - FailOnSharedResource=true

  revisionHistoryLimit: 20

```

Automatic sync can be configured using `spec.syncPolicy.automated`, with pruning and self-healing controlled separately. Argo CD also supports options such as `PruneLast`, `ApplyOutOfSyncOnly`, `FailOnSharedResource`, server-side apply and confirmation-based deletion. (argo-cd.readthedocs.io)

Why these choices?

- `allowEmpty: false`: a rendering error should not delete the entire application.
- `PruneLast=true`: obsolete objects are removed after new resources have been applied and become healthy.
- `ApplyOutOfSyncOnly=true`: reduces unnecessary writes in larger applications.
- `FailOnSharedResource=true`: prevents two applications silently trying to own the same resource.
- `No CreateNamespace=true`: namespaces are created through the controlled platform bootstrap project.

10. Multi-environment ApplicationSet

Use an `ApplicationSet` when several `Applications` have the same structure and only a small number of values vary.

The following generates three applications:

- `rag-dev`
- `rag-staging`
- `rag-prod`

Development and staging use automatic sync. Production remains manual.

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: rag-environments
  namespace: argocd
spec:
  goTemplate: true
  goTemplateOptions:
    - missingkey=error

  generators:
    - list:
        elements:
          - environment: dev
            namespace: rag-dev
            autoSync: "true"

          - environment: staging
            namespace: rag-staging
            autoSync: "true"

          - environment: prod
            namespace: rag-prod
            autoSync: "false"
```

```

template:
  metadata:
    name: "rag-{{ .environment }}"
    labels:
      platform.example.com/team: genai
      platform.example.com/environment: "{{ .environment }}"

  spec:
    project: genai-platform

    source:
      repoURL: git@github.com:example-org/genai-platform-deployments.git
      targetRevision: main
      path: charts/rag-platform
      helm:
        releaseName: rag-platform
        valueFiles:
          - "../environments/{{ .environment }}/values.yaml"
          - "../environments/{{ .environment }}/release-values.yaml"

    destination:
      server: https://kubernetes.default.svc
      namespace: "{{ .namespace }}"

    syncPolicy:
      retry:
        limit: 5
        backoff:
          duration: 10s
          factor: 2
          maxDuration: 3m

      syncOptions:
        - PruneLast=true
        - ApplyOutOfSyncOnly=true
        - FailOnSharedResource=true

    revisionHistoryLimit: 20

templatePatch: |
  {{- if eq .autoSync "true" }}
  spec:
    syncPolicy:
      automated:
        enabled: true
        prune: true
        selfHeal: true
        allowEmpty: false
  {{- end }}

```

templatePatch is useful here because a boolean or complete synchronization-policy object cannot be directly templated as a normal string field. It requires Go templating and must only consume trusted generator input because it can patch arbitrary parts of the generated Application. (argocd.readthedocs.io)

When ApplicationSet genuinely helps

Use it when:

- The same application targets many environments or clusters.
- New environments are added regularly.
- Application definitions are otherwise nearly identical.
- You want generator-driven consistency.

Do not use it merely to replace two small Applications with difficult template logic.

11. Immutable ECR image values

11.1 Release values

environments/dev/release-values.yaml:

```
release:
  id: "2026-07-30.3"
  sourceCommit: "7fb061d9c6276df81d14328a46fd6f33e25324ba"

image:
  repository: >-
    111122223333.dkr.ecr.ap-south-1.amazonaws.com/rag-api
  digest: >-
    sha256:83a6c02a994f5f68167165dbde0b9aaee312f95220f9519dcf8962d772ec2134

model:
  provider: bedrock
  modelId: "anthropic.claude-sonnet-example"
  modelRevision: "2026-07-15"

prompt:
  bundleId: "rag-prompts-2026-07-29.2"
  bundleSha256: >-
    53c1eac68ce913d21505b5dff254282df994e8cf8bfa82b4f00c332b0df1ffdd

embedding:
  modelId: "amazon.titan-embed-text-example"
  revision: "v2"
  dimensions: 1024
  normalization: "l2"

vectorIndex:
  collection: "customer-support-v7"
  schemaVersion: "7"
  snapshotId: "snapshot-2026-07-29-1840"
  embeddingContract: "titan-v2-1024-l2"
```

```
dataset:
  corpusVersion: "support-corpus-2026-07-28"

compatibility:
  applicationContract: "rag-contract-v7"
  minimumIndexSchema: "7"

promotionEvidence:
  ciRunId: "ci-738194"
  evaluationReportUri: >-
    s3://genai-release-evidence/rag-api/2026-07-30.3/report.json
  evaluationReportSha256: >-
    f9dde66b805293388dfb53d7d60543a44c1b759f887f1b943b4a17c9d5cf72e7
  approvalTicket: "GENAI-4217"
```

11.2 Helm image template

```
containers:
  - name: rag-api
    image: >-
      {{ required "image.repository is required" .Values.image.repository
      }}@{{ required "image.digest is required" .Values.image.digest }}
    imagePullPolicy: IfNotPresent
```

Rendered image:

```
111122223333.dkr.ecr.ap-south-1.amazonaws.com/rag-api
@sha256:83a6c02a...
```

A tag can remain as human-readable release metadata, but the workload should use the digest.

11.3 Environment-only values

environments/prod/values.yaml:

```
replicaCount: 6

environment: prod

serviceAccount:
  name: rag-api

resources:
  requests:
    cpu: "1000m"
    memory: "2Gi"
  limits:
    cpu: "2000m"
    memory: "4Gi"

autoscaling:
  enabled: true
```

```
minReplicas: 6
maxReplicas: 30

externalSecrets:
  enabled: true
  secretStoreName: aws-secretsmanager-prod
  remoteSecretKey: /genai/prod/rag-api

ingestion:
  enabled: true
  concurrency: 4

evaluationJob:
  enabled: false
```

This file does not redefine image or model versions. Those belong to the promoted release tuple.

12. Safe promotion workflow

12.1 Step 1: CI publishes the immutable outputs

Assume CI produced:

```
Release: 2026-07-30.3
Image digest: sha256:83a6c02a...
Prompt bundle: rag-prompts-2026-07-29.2
Index contract: rag-contract-v7
Evaluation report: s3://.../report.json
```

The release process must verify:

- Image exists in ECR.
- Digest matches the signed artifact.
- Evaluation report belongs to the same application commit.
- Prompt hash matches the evaluated prompt.
- Embedding and vector-index contracts are compatible.
- Required approvals or quality gates passed.

Argo CD consumes this evidence; it does not reproduce the build and evaluation pipeline.

12.2 Step 2: Promote to development

Exact change:

```
# environments/dev/release-values.yaml

release:
- id: "2026-07-28.6"
+ id: "2026-07-30.3"

image:
- digest: "sha256:5e9d..."
```

```

+ digest: "sha256:83a6c02a..."

  prompt:
- bundleId: "rag-prompts-2026-07-27.4"
+ bundleId: "rag-prompts-2026-07-29.2"

  vectorIndex:
- snapshotId: "snapshot-2026-07-27-1100"
+ snapshotId: "snapshot-2026-07-29-1840"

  promotionEvidence:
- ciRunId: "ci-736002"
+ ciRunId: "ci-738194"

```

After the PR is merged:

1. rag-dev becomes OutOfSync.
2. Automatic sync begins.
3. Argo applies the new desired state.
4. Application health moves through Progressing.
5. It becomes Healthy, or Degraded if deployment health fails.

12.3 Step 3: Promote the exact tuple to staging

Do not independently retype every field.

A release script can copy the complete approved release block:

```

cp environments/dev/release-values.yaml \
  environments/staging/release-values.yaml

git add environments/staging/release-values.yaml
git commit -S -m "Promote release 2026-07-30.3 to staging"

```

The staging PR should verify that the complete tuple matches development:

Application digest	identical
Model revision	identical
Prompt hash	identical
Embedding contract	identical
Vector snapshot	identical
Evaluation evidence	identical

Environment-only settings such as replicas and database endpoints remain different.

12.4 Step 4: Promote to production

```

cp environments/staging/release-values.yaml \
  environments/prod/release-values.yaml

git add environments/prod/release-values.yaml
git commit -S -m "Promote release 2026-07-30.3 to production"

```

Production protections should require:

- Platform-owner approval.
- Application-owner approval.
- Successful configuration-policy checks.
- Verification that staging deployed the same tuple.
- Protected branch.
- CODEOWNERS review.
- No direct push.
- Signed commit or verified merge where supported.
- Change ticket and evidence reference.

After merge, rag-prod becomes OutOfSync, but it does not automatically deploy because production automatic sync is disabled.

An authorized release manager performs:

```
argocd app diff rag-prod
argocd app sync rag-prod
argocd app wait rag-prod --health --sync --timeout 900
```

The sync window can prevent deployment outside the approved period. Sync windows can restrict manual and automated synchronization, with carefully controlled manual override behavior. (argocd.readthedocs.io)

13. Synchronization behavior

13.1 Manual sync

With manual synchronization:

- Git changes are detected.
- The Application becomes OutOfSync.
- No resources are changed until an authorized user syncs.

Best suited to:

- Production.
 - Regulated environments.
 - High-risk migrations.
 - Infrastructure controllers.
 - Sensitive model or index changes.
-

13.2 Automated sync

With automatic synchronization:

- Git change is detected.
- Argo automatically applies the desired state.
- CI does not need to invoke Argo or Kubernetes.

Best suited to:

- Development.
 - Ephemeral environments.
 - Lower-risk staging environments.
 - Cluster add-ons with strongly controlled configuration repositories.
-

13.3 prune

Without pruning:

```
Resource removed from Git
      ↓
Resource may remain in Kubernetes
```

With pruning:

```
Resource removed from Git
      ↓
Argo deletes the live resource
```

Safe controls include:

- `allowEmpty: false`
- `PruneLast=true`
- `Prune=confirm` for high-risk resources
- `Prune=false` for externally managed resources
- Finalizers for resources requiring controlled cleanup
- PR checks detecting unexpectedly large resource deletion

Automated pruning is intentionally separate from automated synchronization because deletion carries higher risk. (argo-cd.readthedocs.io)

13.4 selfHeal

With self-healing enabled:

```
Manual kubectl edit
      ↓
Argo detects live drift
      ↓
Argo restores Git version
```

Use it for:

- Image references.
- Security contexts.
- Service selectors.
- Environment variables.
- Network policies.
- Resource limits where Git is the owner.

Do not use it blindly for:

- Fields changed by autoscalers.
- Mutating admission controllers.
- Operator-managed status or defaults.
- Dynamic fields intentionally owned outside Git.

13.5 Sync status versus health status

Sync status

Answers:

| Does live configuration match desired configuration?

Typical values:

- Synced
- OutOfSync
- Unknown

Health status

Answers:

| Is the deployed application operating as Kubernetes expects?

Common values include:

- Healthy
- Progressing
- Degraded
- Missing
- Suspended
- Unknown

Argo CD performs built-in health assessment for standard Kubernetes resources and can support custom health checks for CRDs. (argo-cd.readthedocs.io)

Important combinations

Sync	Health	Interpretation
Synced	Healthy	Desired state is applied and healthy
Synced	Progressing	Desired state is applied; rollout is ongoing
Synced	Degraded	Desired state is applied but workload failed
OutOfSync	Healthy	Old live version may still be healthy
OutOfSync	Degraded	Configuration differs and workload is unhealthy
Unknown	Unknown	Argo cannot determine desired or live state reliably

Never treat Synced as equivalent to Healthy.

13.6 Useful sync options

PruneLast=true

Delete obsolete resources only after the other waves complete successfully.

ApplyOutOfSyncOnly=true

Apply only resources that differ.

Useful for large applications, but selective operations require caution because hooks do not run during selective sync, and selective sync history has limitations. (argo-cd.readthedocs.io)

FailOnSharedResource=true

Fail if another Argo application already owns a resource.

CreateNamespace=true

Convenient for low-risk environments. In a restricted production design, create namespaces through a separate platform application instead.

ServerSideApply=true

Useful when:

- Manifests are large.
- Multiple managers legitimately own different fields.
- Server-side ownership is understood.

Do not enable it casually because it changes field-management semantics.

SkipDryRunOnMissingResource=true

Useful when a CRD and its custom resource appear together or when another controller creates the CRD.

Prefer installing CRDs separately rather than masking unexpected missing-CRD problems.

RespectIgnoreDifferences=true

Makes synchronization respect configured ignored fields, rather than using ignore rules only for diff reporting.

Use only after confirming which controller owns the field.

Replace=true or force-related options

Potentially destructive. They can recreate resources and cause downtime. Avoid as a general default.

14. Hooks, phases and sync waves

Argo CD supports hook phases such as PreSync, Sync, PostSync, SyncFail, PreDelete, and PostDelete. Waves order resources within phases using integer annotations; lower waves run first, and pruning reverses wave order. Hooks are often Jobs or Argo Workflows. (argo-cd.readthedocs.io)

14.1 Recommended resource ordering

Wave -5	Namespace and platform prerequisites
Wave -4	CRDs
Wave -3	Operators/controllers

```
Wave -2 SecretStore and service accounts
Wave -1 ExternalSecret and configuration
PreSync Compatibility and migration Jobs
Wave 0 API and ingestion workloads
Wave 1 Services and traffic resources
PostSync Smoke validation
```

Do not place CRDs and all applications in the same project merely to achieve ordering. Cluster-level components should generally be managed through a restricted platform project.

14.2 ExternalSecret wave

```
apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: rag-api-secrets
  annotations:
    argocd.argoproj.io/sync-wave: "-1"
spec:
  refreshPolicy: Periodic
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secretsmanager-prod
    kind: SecretStore
  target:
    name: rag-api-secrets
    creationPolicy: Owner
  data:
    - secretKey: MODEL_PROVIDER_API_KEY
      remoteRef:
        key: /genai/prod/rag-api
        property: modelProviderApiKey
```

A lower wave applies the ExternalSecret before the Deployment, but the External Secrets controller still works asynchronously. For strict ordering, add a bounded PreSync check that verifies the generated Kubernetes Secret exists and contains the expected keys.

14.3 PreSync compatibility check

```
apiVersion: batch/v1
kind: Job
metadata:
  name: rag-release-compatibility-check
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/sync-wave: "-1"
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation,HookSucceeded
spec:
  backoffLimit: 1
  ttlSecondsAfterFinished: 600
  template:
    spec:
```

```

restartPolicy: Never
serviceAccountName: rag-release-validator
containers:
  - name: validator
    image: >-
      111122223333.dkr.ecr.ap-south-1.amazonaws.com/release-validator
      @sha256:4e3a...
    args:
      - validate
      - --application-
contract={{ .Values.compatibility.applicationContract }}
      - --index-schema={{ .Values.vectorIndex.schemaVersion }}
      - --embedding-
contract={{ .Values.vectorIndex.embeddingContract }}
      - --prompt-sha={{ .Values.prompt.bundleSha256 }}

```

This check does not rebuild or reevaluate the model. It confirms that the already-approved release tuple refers to compatible runtime artifacts.

14.4 Database migration hook

```

apiVersion: batch/v1
kind: Job
metadata:
  name: rag-database-migration
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/sync-wave: "0"
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation,HookSucceeded
spec:
  backoffLimit: 1
  activeDeadlineSeconds: 600
  template:
    spec:
      restartPolicy: Never
      serviceAccountName: rag-api
      containers:
        - name: migration
          image: >-
            {{ .Values.image.repository }}@{{ .Values.image.digest }}
          command:
            - python
            - -m
            - app.database.migrate

```

Keep migrations backward-compatible with the previous application version. Follow your existing database rollback and expand/migrate/contract notes rather than relying on Argo CD to reverse a destructive schema operation.

14.5 PostSync smoke check

```
apiVersion: batch/v1
kind: Job
metadata:
  name: rag-postsync-smoke
  annotations:
    argocd.argoproj.io/hook: PostSync
    argocd.argoproj.io/hook-delete-policy: HookSucceeded
spec:
  backoffLimit: 1
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: smoke
          image: >-
            11112223333.dkr.ecr.ap-south-1.amazonaws.com/smoke-runner
            @sha256:cc92...
          args:
            - --url=http://rag-api.rag-prod.svc.cluster.local/health/ready
            - --expected-release={{ .Values.release.id }}
```

A PostSync failure marks the synchronization operation failed, but it does not automatically undo every previously applied resource. Recovery must still be explicitly designed.

15. Safe handling of CRDs

A common failure is:

```
Custom resource applied
      before
its CRD exists
```

Preferred pattern:

```
platform-addons Application
├── External Secrets CRDs/controller
├── Argo Rollouts CRDs/controller
└── policy-controller CRDs/controller

rag-platform Application
├── ExternalSecret
├── Rollout
└── application workloads
```

This gives:

- Separate permissions.
- Independent upgrades.
- Smaller failure domains.
- Clearer ownership.

- No application project permission to install cluster-wide CRDs.

Use `SkipDryRunOnMissingResource=true` only when the dependency is deliberate and understood, not as a universal fix.

16. Sync windows

Sync windows can define:

- An allowed deployment period.
- A denied freeze period.
- Matching applications, namespaces or clusters.
- Whether manual sync override is allowed.

Example production freeze:

```
syncWindows:
  - kind: deny
    schedule: "0 0 20 12 *"
    duration: 336h
    applications:
      - rag-prod
    namespaces:
      - rag-prod
    manualSync: false
```

This example represents a long holiday freeze starting December 20. In practice, review cron expressions carefully and test them before relying on them for governance.

Sync windows are a guardrail, not a full approval system. Git protection, Argo RBAC and change-management controls must still be applied.

17. Diff customization

17.1 Why drift can reappear

A resource may remain `OutOfSync` because:

- An HPA modifies replicas.
- An admission controller injects labels or sidecars.
- Kubernetes normalizes values.
- A controller reorders a list.
- An operator owns part of the object.
- Helm renders nondeterministic content.
- Two controllers fight over the same field.

First identify the field owner:

```
kubectl get deployment rag-api \
  -n rag-prod \
  -o yaml \
  --show-managed-fields
```

Then narrow the ignore rule.

17.2 Narrow ignore example

```
spec:
  ignoreDifferences:
    - group: apps
      kind: Deployment
      name: rag-api
      namespace: rag-prod
      jsonPointers:
        - /spec/replicas
```

This may be appropriate when an HPA owns the replica count.

Argo CD supports application-level ignore rules using JSON pointers, JQ expressions or managed-field managers. Broad ignore rules can hide real unauthorized changes. (argo-cd.readthedocs.io)

Avoid:

```
ignoreDifferences:
  - group: "*"
    kind: "*"
    jqPathExpressions:
      - .
```

That effectively destroys meaningful drift detection.

17.3 Diff only versus sync behavior

By default, an ignored field may be omitted from comparison but still overwritten during synchronization.

To make synchronization also respect the ignore rule:

```
syncOptions:
  - RespectIgnoreDifferences=true
```

Use this only when another trusted controller genuinely owns that field.

18. Rollback and recovery

18.1 Preferred rollback: Git revert

Suppose production commit:

```
4ef92ab Promote release 2026-07-30.3 to production
```

Revert it:

```
git revert 4ef92ab
git push origin rollback/2026-07-30.3
```

After approval and merge:

```
argocd app diff rag-prod
argocd app sync rag-prod
argocd app wait rag-prod --health --sync
```

Advantages:

- Git again represents reality.
- The rollback is reviewed.
- The reason is recorded.
- Argo does not continuously attempt to redeploy the bad version.
- Recovery survives Argo CD restoration.

18.2 Argo CD history rollback

Argo CD can roll back to a historical deployment:

```
argocd app history rag-prod
argocd app rollback rag-prod 17
```

This is useful when:

- Immediate operational restoration is necessary.
- Git revert approval is temporarily delayed.
- You need to restore the last known manifest set rapidly.

However, Argo CD does not permit a normal history rollback while automated sync is enabled, because reconciliation would immediately try to restore the current Git state. (argo-cd.readthedocs.io)

For an automatic-sync application:

1. Change the source-controlled Application/ApplicationSet policy.
2. Disable automated synchronization.
3. Perform the emergency rollback.
4. Immediately create a Git revert.
5. Re-enable automated sync after Git represents the recovered state.

Do not manually patch an Application generated by an ApplicationSet; the ApplicationSet controller will normally restore the generated specification.

18.3 Accidental prune

Response:

1. Stop further synchronization if more deletion is possible.
2. Identify the Git commit that removed the resource.
3. Restore the manifest through Git.
4. Sync the application.
5. Restore persistent data from the relevant backup if deletion removed storage.
6. Review why the deletion passed PR and prune controls.

Prevention:

- `allowEmpty: false`
 - `PruneLast=true`
 - `Prune=confirm`
 - Protected production directories.
 - Change-size checks.
 - Resource finalizers where appropriate.
 - Separate persistent data lifecycle from application lifecycle.
-

18.4 Bad manifest

Symptoms:

- Manifest-generation error.
- Invalid Kubernetes schema.
- Admission-policy denial.
- Missing value.
- Helm template failure.

Recovery:

```
argocd app get rag-prod
argocd app manifests rag-prod
argocd app diff rag-prod
argocd app refresh rag-prod --hard
```

Then revert or fix the Git commit.

Do not bypass a valid admission-policy rejection simply to make Argo show Synced.

18.5 Stuck hook

Inspect:

```
argocd app get rag-prod --show-operation
kubectl get jobs -n rag-prod
kubectl describe job rag-database-migration -n rag-prod
kubectl logs job/rag-database-migration -n rag-prod
```

Terminate the Argo operation when necessary:

```
argocd app terminate-op rag-prod
```

Then:

- Fix the cause.
 - Delete a stale hook only after preserving logs.
 - Ensure `BeforeHookCreation` is set if the next sync must recreate a fixed-name hook.
 - Re-run the full sync, not a selective sync that skips hooks.
-

18.6 Git repository unavailable

Existing workloads continue running because Kubernetes does not depend on Git for every request.

However:

- Argo cannot reliably fetch or render the newest desired state.
- Refresh or sync may fail.
- Application status may become Unknown.
- Repository-server errors should alert the platform team.

Response:

- Do not perform speculative synchronization from uncertain state.
 - Restore Git connectivity or credentials.
 - Verify the target revision.
 - Hard-refresh the application.
 - Compare Git and live state before resuming deployment.
-

18.7 Failed migration

Do not repeatedly force-sync a failed destructive migration.

Instead:

1. Stop the operation.
 2. Preserve migration logs.
 3. Determine whether the schema was partially changed.
 4. Follow the existing database recovery plan.
 5. Restore a compatible application and schema combination.
 6. Commit the recovered desired state to Git.
 7. Correct the hook before another promotion.
-

19. GenAI release management

A normal web service release may be represented by one image digest.

A GenAI release is usually a compatible tuple:

Application image
Model or provider deployment
Prompt bundle
Embedding model
Vector-index schema
Vector-index snapshot
Corpus/dataset version
Reranker version
Tool/API contract
Evaluation evidence

19.1 Why the tuple matters

Consider:

Application expects 1536-dimensional vectors
Index contains 1024-dimensional vectors

The image may deploy successfully, but retrieval fails.

Another example:

Prompt requires tool schema v4
Agent service exposes tool schema v3

Kubernetes health may be green while agent calls fail.

Therefore, promotion must validate compatibility, not merely update an image.

19.2 Recommended Git-managed fields

```
application:
  digest: sha256:...

model:
  provider: bedrock
  modelId: ...
  revision: ...

prompt:
  bundleId: ...
  sha256: ...

embedding:
  modelId: ...
  dimensions: 1024
  normalization: l2

vectorIndex:
  collection: ...
  schemaVersion: "7"
  snapshotId: ...
  embeddingContract: titan-v2-1024-l2

reranker:
  modelId: ...
  revision: ...

evaluation:
  reportUri: s3://...
  reportSha256: ...
  gatePolicyVersion: rag-gate-v5
```

These are version references and contracts, not secret values.

19.3 Preventing incompatible promotion

Use layered controls:

Pull-request validation

Validate:

- Required fields.
- Allowed model IDs.
- Embedding dimensions.
- Prompt hash format.
- Index schema compatibility.
- Evidence URI and digest.
- Promotion source environment.
- Image registry and digest format.

PreSync compatibility Job

Check runtime metadata:

- Vector collection exists.
- Schema matches.
- Embedding dimensions match.
- Prompt bundle exists and hash matches.
- Provider model or deployment ID is enabled.
- Required tool contract exists.

Admission policy

Enforce what Kubernetes can verify:

- ECR registry allowlist.
- Digest-pinned image.
- Signature or attestation.
- Required security context.
- Required labels.
- No privileged container.

Runtime progressive delivery

Use trusted runtime metrics to detect:

- HTTP error increase.
- Latency regression.
- Provider throttling.
- Retrieval failures.
- Tool-call failures.
- Token-cost anomalies.

Do not let a weak or low-volume semantic metric independently promote production.

19.4 API, ingestion and evaluation ordering

A practical deployment sequence:

1. Secret and configuration references
2. Compatibility validation

3. Backward-compatible schema migration
4. API deployment
5. Ingestion worker deployment
6. Traffic exposure
7. PostSync smoke check

For an index rebuild:

```
New index built outside serving path
      ↓
Index validation completed
      ↓
Release tuple references new snapshot
      ↓
API canary reads new index
      ↓
Traffic gradually increases
```

Do not make a production API wait indefinitely for an ingestion Job that can take hours. Treat large index construction as a separate controlled workflow whose completed output becomes a release input.

20. Secrets patterns

20.1 Why base64 Kubernetes Secrets are not secure

This is encoding:

```
data:
  password: cGFzc3dvcmQ=
```

Anyone with repository access can decode it:

```
echo cGFzc3dvcmQ= | base64 --decode
```

Base64 does not provide encryption or access control.

20.2 External Secrets Operator with AWS Secrets Manager

Current External Secrets APIs use resources such as:

- SecretStore
- ClusterSecretStore
- ExternalSecret

The operator synchronizes values from external providers into Kubernetes Secrets. AWS provider documentation recommends fine-grained roles and supports AWS Secrets Manager and Parameter Store. ([External Secrets](#))

SecretStore

```
apiVersion: external-secrets.io/v1
kind: SecretStore
```

```

metadata:
  name: aws-secretsmanager-prod
  namespace: rag-prod
spec:
  provider:
    aws:
      service: SecretsManager
      region: ap-south-1
      role: arn:aws:iam::111122223333:role/rag-prod-secret-reader

```

ExternalSecret

```

apiVersion: external-secrets.io/v1
kind: ExternalSecret
metadata:
  name: rag-api-secrets
  namespace: rag-prod
spec:
  refreshPolicy: Periodic
  refreshInterval: 1h

  secretStoreRef:
    name: aws-secretsmanager-prod
    kind: SecretStore

  target:
    name: rag-api-secrets
    creationPolicy: Owner
    deletionPolicy: Retain

  data:
    - secretKey: DATABASE_PASSWORD
      remoteRef:
        key: /genai/prod/rag-api
        property: databasePassword

    - secretKey: MODEL_PROVIDER_API_KEY
      remoteRef:
        key: /genai/prod/rag-api
        property: modelProviderApiKey

```

Git contains:

- Secret name.
- Remote key path.
- Refresh behavior.
- Target Kubernetes Secret name.

Git does not contain the secret value.

20.3 Secret rotation

Rotation flow:

```
Update value in AWS Secrets Manager
      ↓
External Secrets refreshes target Secret
      ↓
Workload reloads or restarts safely
```

Depending on the application:

- Read the Secret on each connection.
- Use a reload sidecar/controller.
- Trigger a checksum-based rollout.
- Support overlapping old and new credentials during rotation.

Do not commit a new secret value merely to cause redeployment.

20.4 EKS Pod Identity and IRSA

Argo CD manages the Kubernetes ServiceAccount, but the AWS identity binding must follow your existing infrastructure design.

With EKS Pod Identity, an AWS-side association maps:

```
Cluster + namespace + service account
                to
                IAM role
```

Unlike IRSA, EKS Pod Identity does not require an IAM-role annotation on the ServiceAccount. ([AWS Documentation](#))

For example, Git can manage:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: rag-api
  namespace: rag-prod
```

Terraform manages:

```
rag-prod/rag-api ServiceAccount
      ↓
EKS Pod Identity association
      ↓
Least-privilege IAM role
```

Do not let the application chart create or modify arbitrary AWS IAM roles.

20.5 Sealed Secrets

Sealed Secrets encrypt secret material into a custom resource that only the target cluster's controller can decrypt.

Useful when:

- An encrypted secret must be stored in Git.
- External secret infrastructure is unavailable.
- Cluster-bound encryption is acceptable.

Costs:

- Certificate/key lifecycle.
- Disaster-recovery dependency on the sealing private key.
- Resealing when moving clusters.
- Rotation management.

For AWS-centric production platforms, External Secrets plus Secrets Manager is often simpler operationally.

21. Production security and governance

21.1 SSO and RBAC example

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-rbac-cm
  namespace: argocd
data:
  policy.default: role:authenticated

  policy.csv: |
    p, role:authenticated, applications, get, genai-platform/*, allow

    p, role:genai-dev, applications, sync,
    genai-platform/rag-dev, allow

    p, role:genai-release, applications, sync,
    genai-platform/rag-staging, allow

    p, role:genai-prod-release, applications, sync,
    genai-platform/rag-prod, allow

    g, genai-developers, role:genai-dev
    g, genai-release-managers, role:genai-release
    g, genai-production-release, role:genai-prod-release
```

In a real configuration, keep each policy line syntactically complete on one line.

Validate RBAC:

```
argocd admin settings rbac validate \
  --policy-file policy.csv

argocd admin settings rbac can \
  genai-production-release \
```

```
sync \
applications \
genai-platform/rag-prod
```

Argo CD's RBAC evaluator has specific glob and matching behavior, so test authorization rather than assuming wildcard semantics. (argo-cd.readthedocs.io)

21.2 Separation of duties

A strong model:

Role	Capability
Developer	Change application code
CI system	Build, test, sign and publish artifacts
Release automation	Propose deployment-repository PR
Application owner	Approve functional release
Platform owner	Approve production platform change
Argo CD	Reconcile approved Git state
Auditor	Read Git and Argo audit records

A developer should not be able to:

1. Build an unreviewed image.
2. Change production deployment state.
3. Approve the change.
4. Sync it to production.

21.3 Repository and cluster credentials

Protect:

- Git deploy keys and GitHub App keys.
- Cluster access Secrets.
- Helm/OCI repository credentials.
- Webhook secrets.
- Notification credentials.
- OIDC client secrets.

Controls:

- Encrypt Kubernetes Secrets at rest.
- Restrict access to the argocd namespace.
- Use dedicated credentials.
- Rotate credentials.
- Prefer read-only Git credentials.
- Avoid credential sharing between trust boundaries.
- Restrict outbound access from the repository server.
- Do not install arbitrary config-management plugins.

21.4 Pod Security Admission

Use namespace labels to enforce modern Pod Security Standards:

```
apiVersion: v1
kind: Namespace
metadata:
  name: rag-prod
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

Pod Security Admission is the built-in replacement for the removed PodSecurityPolicy.
([Kubernetes](#))

21.5 Required security context

```
securityContext:
  runAsNonRoot: true
  seccompProfile:
    type: RuntimeDefault

containers:
- name: rag-api
  securityContext:
    allowPrivilegeEscalation: false
    readOnlyRootFilesystem: true
    capabilities:
      drop:
        - ALL
```

Kubernetes security contexts control privilege, user identity, filesystem and capability behavior.
([Kubernetes](#))

21.6 Default-deny NetworkPolicy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: rag-prod
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress
```

Then add explicit policies for:

- Ingress controller to API.

- API to Postgres.
- API to Redis.
- API to vector database.
- API to approved model-provider endpoints.
- DNS.
- Monitoring.
- Ingestion worker to storage/index services.

A compatible network plugin must enforce `NetworkPolicy`; creating the object alone is insufficient when the CNI does not implement enforcement. ([Kubernetes](#))

21.7 Policy as code

Use Kyverno or OPA Gatekeeper to reject resources that violate platform policy.

Recommended checks:

- Images must come from approved ECR accounts.
- Image references must contain a digest.
- Images must be signed by approved CI identities.
- Required attestations must exist.
- Containers must run as non-root.
- Privilege escalation must be disabled.
- Required CPU and memory requests must exist.
- Host networking and host paths must be denied.
- Production resources must have owner and data-classification labels.

Kyverno supports image verification, digest requirements and signature/attestation checks. Gatekeeper enforces admission policies using OPA-based constraints. ([Kyverno](#))

Admission policy provides defence in depth:

```
Malicious or incorrect Git commit
      ↓
Argo attempts apply
      ↓
Admission controller rejects violation
      ↓
Argo reports failed sync
```

Argo being authorized to sync does not mean every manifest should automatically be accepted.

22. Argo CD and related tools

22.1 Argo CD and Helm

Argo CD uses Helm primarily as a manifest renderer:

```
Chart + values
      ↓
helm template-like rendering
```

```
↓
Kubernetes manifests
↓
Argo reconciliation
```

Argo CD, rather than Helm's release state, tracks and reconciles the generated Kubernetes resources.

Useful when:

- The application already has a reusable chart.
- Environments differ through values.
- You want Argo reconciliation and Helm templating together.

Unnecessary complexity when:

- A tiny application has a few static manifests.
- Templates become unreadable condition trees.
- Every environment overrides most of the chart.

22.2 Argo CD and Argo Rollouts

Argo CD answers:

| Which rollout specification should exist?

Argo Rollouts answers:

| How should a new ReplicaSet progressively receive traffic?

```
Git Rollout manifest
↓
Argo CD applies desired Rollout
↓
Argo Rollouts controller manages canary/blue-green progression
```

Use Rollouts when:

- Runtime canary analysis is required.
- Traffic splitting is available.
- Rollback should react to reliable runtime metrics.
- Deployment risk justifies the additional controller.

Avoid it when:

- A normal rolling Deployment is sufficient.
 - Traffic routing cannot support canary separation.
 - Metrics are unreliable.
 - The team cannot operate another controller and CRD.
-

22.3 Trusted Argo Rollouts analysis

AnalysisTemplate

```
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: rag-api-runtime-analysis
  namespace: rag-prod
spec:
  metrics:
    - name: sufficient-request-volume
      interval: 2m
      count: 5
      failureLimit: 1
      successCondition: result[0] >= 500
      failureCondition: result[0] < 500
      provider:
        prometheus:
          address: http://prometheus.monitoring.svc.cluster.local:9090
          query: |
            sum(
              increase(
                http_requests_total{
                  namespace="rag-prod",
                  app="rag-api",
                  rollout_phase="canary"
                }[5m]
              )
            )

    - name: successful-request-ratio
      interval: 2m
      count: 5
      failureLimit: 2
      successCondition: result[0] >= 0.995
      failureCondition: result[0] < 0.990
      provider:
        prometheus:
          address: http://prometheus.monitoring.svc.cluster.local:9090
          query: |
            (
              sum(
                rate(
                  http_requests_total{
                    namespace="rag-prod",
                    app="rag-api",
                    rollout_phase="canary",
                    status!="5.."
                  }[5m]
                )
              )
            )
          /
```

```

        clamp_min(
            sum(
                rate(
                    http_requests_total{
                        namespace="rag-prod",
                        app="rag-api",
                        rollout_phase="canary"
                    }[5m]
                )
            ),
            0.001
        )
    )

```

Rollout steps

```

strategy:
  canary:
    stableService: rag-api-stable
    canaryService: rag-api-canary

    steps:
      - setWeight: 5
      - pause:
          duration: 5m

      - analysis:
          templates:
            - templateName: rag-api-runtime-analysis

      - setWeight: 20
      - pause: {}

      - setWeight: 50
      - pause:
          duration: 10m

```

Behavior:

- Failed analysis aborts the canary.
- Successful analysis proceeds to the next step.
- The indefinite pause after 20% requires an authorized promotion decision.
- An inconclusive or unavailable metric must not be interpreted as success.

Argo Rollouts can abort when analysis fails and can pause when metrics cannot provide a reliable success/failure result. ([Argo Rollouts](#))

What makes a metric trusted?

- Produced server-side, not by the candidate pod.
- Protected Prometheus access.
- Immutable labels or routing identifiers.
- Minimum traffic requirement.
- Stable query and evaluation window.
- Known behavior for empty, NaN and missing results.

- Multiple signals: errors, latency, saturation and business failures.
 - No automatic production promotion from a single subjective LLM-quality score.
 - Existing offline/golden evaluation passed before production promotion.
-

22.4 Argo CD Image Updater

Image Updater is a separate component that watches registries and proposes or writes updated image references.

Useful for:

- Development environments.
- Frequent patch updates.
- Internal services with automated release policies.
- Opening Git-based update PRs.

For GitOps consistency, prefer Git write-back or PR-based behavior over direct modification of the Argo Application. Current Image Updater documentation describes Git write-back and newer PR-based workflows. ([Argo CD Image Updater](#))

Avoid automatic production updates when:

- Model, prompt and index must be promoted as one tuple.
- Evaluation evidence is mandatory.
- Every release requires human approval.
- “Newest image” does not necessarily mean “approved release.”

For your GenAI platform, a controlled release PR is generally safer than independent image auto-updating.

22.5 App-of-apps pattern

A root Application manages child Application manifests:

```
root Application
├── platform-addons Application
├── observability Application
├── rag-dev Application
├── rag-staging Application
└── rag-prod Application
```

Useful for:

- Bootstrapping a known set of platform applications.
- Organizing one cluster.
- Managing applications declaratively.

Risks:

- Root project may become excessively privileged.
 - Deleting the root can cascade.
 - Cross-Application ordering is often overestimated.
 - Application health does not automatically provide a perfect dependency engine.
-

22.6 ApplicationSet versus app-of-apps

Requirement	Better fit
Generate one Application per cluster	ApplicationSet
Generate one Application per environment directory	ApplicationSet
Bootstrap a small fixed collection	App-of-apps
Discover clusters from Argo registration	ApplicationSet
Apply explicit root-managed platform layout	App-of-apps
Complex workflow dependency orchestration	Neither; use a workflow system

ApplicationSet generators can create Applications from lists, Git directories/files, registered clusters and combinations of generators. Security must be carefully controlled when developers influence generated project or destination fields. (argo-cd.readthedocs.io)

23. Observability and operations

23.1 Important metrics

Monitor:

Application state

- Applications not Synced.
- Applications not Healthy.
- Sync-operation failures.
- Long-running synchronization.
- Repeated self-heal.
- Orphan-resource warnings.

Repository server

- Git fetch failures.
- Git request duration.
- Manifest-generation duration.
- Pending repository requests.
- Rendering timeouts.
- Memory and temporary-disk usage.

Application controller

- Reconciliation latency.
- Work queue depth.
- Kubernetes API errors.
- Redis errors.
- Number of managed applications.
- Operation-processing saturation.

ApplicationSet controller

- Reconciliation duration.
- Generated Applications not up to date.

- Generator errors.

Argo CD exposes Prometheus metrics for controllers, the repository server, ApplicationSet and notifications, and provides ServiceMonitor examples. (argo-cd.readthedocs.io)

23.2 Recommended alerts

Actionable:

```
Production application Degraded for > 5 minutes
Production sync failed
Production OutOfSync for > 15 minutes after approved merge
Repository fetch failing for > 10 minutes
Application controller cannot reach EKS API
Repeated self-heal more than N times in 15 minutes
PreSync migration or compatibility Job failed
Argo CD component unavailable
```

Usually noisy:

```
Every successful development sync
Every brief Progressing status
Every expected HPA replica change
Every repository poll
Every transient one-second OutOfSync state
```

Route by labels such as:

- Team.
- Environment.
- Service.
- Criticality.
- Project.

Argo CD supports notification triggers and templates for application events, including sync failures and degraded states. (argo-cd.readthedocs.io)

23.3 High availability

For production:

- Run HA controller/server/repository components as appropriate.
- Spread replicas across nodes and availability zones.
- Use PodDisruptionBudgets.
- Size repository server memory and temporary storage.
- Monitor Redis.
- Protect the Argo namespace.
- Avoid a single shared Argo instance becoming an organization-wide failure domain.

The official HA manifests require multiple nodes because of anti-affinity rules, and the repository server may need explicit scaling and storage planning. (argo-cd.readthedocs.io)

23.4 Backup and disaster recovery

Back up:

- Application, AppProject, and ApplicationSet resources.
- Argo CD ConfigMaps.
- Repository credential Secrets.
- Cluster registration Secrets.
- OIDC and RBAC configuration.
- Notification configuration.
- Application signing keys or certificates where applicable.
- Sealed Secrets private key, when used.

Argo CD provides administrative export/import capabilities for disaster recovery, and its upgrade guidance recommends backing up settings before upgrades. (argo-cd.readthedocs.io)

A recovery sequence:

```
Recreate EKS/platform namespace
    ↓
Install pinned Argo CD version
    ↓
Restore Argo configuration and credentials
    ↓
Apply root Application/ApplicationSet
    ↓
Argo rebuilds desired state from Git
    ↓
Verify before enabling automatic pruning
```

Do not immediately enable broad pruning during DR until Argo has rediscovered all expected live resources and ownership.

23.5 Upgrades

Before an Argo CD upgrade:

1. Read every intermediate upgrade note.
 2. Check removed and deprecated flags.
 3. Back up configuration.
 4. Test repository rendering.
 5. Verify custom health and diff rules.
 6. Test SSO and RBAC.
 7. Test ApplicationSet generation.
 8. Confirm rollback package/version.
 9. Upgrade non-production first.
 10. Observe reconciliation load after upgrade.
-

24. Useful commands

Application inspection

```
argocd app list
argocd app get rag-prod
argocd app get rag-prod --show-operation
argocd app resources rag-prod
argocd app manifests rag-prod
argocd app diff rag-prod
```

Refresh and synchronization

```
argocd app refresh rag-prod
argocd app refresh rag-prod --hard

argocd app sync rag-prod
argocd app sync rag-prod --prune

argocd app wait rag-prod \
  --sync \
  --health \
  --timeout 900
```

History and rollback

```
argocd app history rag-prod
argocd app rollback rag-prod 17
```

Stop a synchronization operation

```
argocd app terminate-op rag-prod
```

Projects

```
argocd proj list
argocd proj get genai-platform
argocd proj windows list genai-platform
```

Repositories and clusters

```
argocd repo list
argocd repo get git@github.com:example-org/genai-platform-deployments.git

argocd cluster list
```

ApplicationSet

```
kubectl get applicationsets -n argocd
kubectl describe applicationset rag-environments -n argocd
```

```
kubectl get applications -n argocd \
  -l app.kubernetes.io/instance=rag-environments
```

Live Kubernetes investigation

```
kubectl get applications -n argocd
kubectl describe application rag-prod -n argocd

kubectl get events -n rag-prod \
  --sort-by=.lastTimestamp

kubectl get deployment,pods,job -n rag-prod
kubectl rollout status deployment/rag-api -n rag-prod

kubectl logs deployment/argocd-repo-server -n argocd
kubectl logs statefulset/argocd-application-controller -n argocd
```

25. Troubleshooting guide

Symptom	Likely causes	Investigation and response
OutOfSync continuously returns	HPA, mutating webhook, defaulted field, nondeterministic Helm output	Inspect diff and managed fields; fix ownership or add narrow ignore rule
Synced but Degraded	Workload applied but readiness, Job or Rollout failed	Inspect resource tree, events, probes and pod logs
Stuck Progressing	Deployment unavailable, custom resource lacks correct health assessment, hook waiting	Inspect resource health; add custom health logic only when required
Unknown	Git, repository server, cluster API or comparison failure	Inspect Application conditions and controller/repository logs
Repository authentication error	Expired token, bad deploy key, missing known host, TLS failure	Rotate credential, test repository access and verify host keys
Manifest-generation error	Helm syntax, missing values, bad dependency, renderer timeout	Render locally with the same values and inspect repo-server logs
Missing CRD	Custom resource deployed before platform controller/CRD	Install CRD through platform Application; avoid blanket skip-dry-run
Permission denied during sync	AppProject restriction, Kubernetes RBAC or admission policy	Inspect Argo project, destination credentials and admission response
Unexpected prune	Resource removed/renamed, Application path changed, empty rendering	Stop sync; restore Git; use <code>allowEmpty: false</code> and confirmation
Failed migration hook	Database connectivity, incompatible schema, partial migration	Preserve logs; stop retries; follow DB recovery plan
Sync loop	Two controllers own same field	Inspect <code>managedFields</code> ; establish one owner or precise ignore behavior

Secret missing	ESO controller unavailable, IAM denied, wrong key/property	Inspect ExternalSecret status, operator logs and AWS permissions
Production will not sync	Sync window, RBAC or project restriction	Check project window and user authorization; do not bypass silently
ApplicationSet change has no effect	Generator error, template error or manual child edit overwritten	Inspect ApplicationSet conditions and generated Application
Image pull failure	Bad digest, cross-account ECR permission, missing node/pod access	Verify digest, repository policy and image pull events
Rollout paused	Manual pause, inconclusive analysis or insufficient traffic	Inspect Rollout and AnalysisRun; never promote without trustworthy evidence

26. Hands-on exercises

Exercise 1: Detect and self-heal drift

1. Deploy rag-dev with automatic sync and self-healing.
2. Manually change an environment variable:

```
kubectl set env deployment/rag-api \
  -n rag-dev \
  LOG_LEVEL=debug
```

1. Observe:

```
argocd app get rag-dev
argocd app diff rag-dev
```

1. Confirm Argo restores the Git value.
2. Repeat after disabling selfHeal and compare behavior.

Learning goal

Understand that automatic sync from a new Git revision and self-healing of live drift are related but separate behaviors.

Exercise 2: Promote one immutable tuple

1. Create release 2026-07-30.3.
2. Update only environments/dev/release-values.yaml.
3. Confirm automatic development synchronization.
4. Copy the exact release file to staging.
5. Verify the rendered image digest:

```
kubectl get deployment rag-api \
  -n rag-staging \
  -o jsonpath='{.spec.template.spec.containers[0].image}'
```

1. Open a production PR containing the exact same tuple.

2. After merge, verify that production is OutOfSync but unchanged.
3. Perform a manual diff and sync.

Learning goal

Separate artifact creation from environment promotion and preserve release identity.

Exercise 3: Build a safe ordering failure

1. Add an ExternalSecret in wave -1.
2. Add a Deployment in wave 0.
3. Temporarily configure an invalid AWS remote-secret key.
4. Observe whether the ExternalSecret application itself becomes healthy before the generated Secret exists.
5. Add a bounded PreSync validation Job.
6. Confirm the deployment is blocked before application pods start.

Learning goal

Understand that sync-wave application order does not automatically guarantee asynchronous controller completion.

27. Senior-level design principles

1. **GitOps is reconciliation, not merely deployment from Git.**
 2. **CI publishes artifacts; Git approves desired deployment state.**
 3. **Production should use immutable digests and controlled PR promotion.**
 4. **A GenAI release is a compatibility tuple, not one container image.**
 5. **AppProjects are security boundaries, not organizational labels.**
 6. **Automatic sync is not equivalent to automatic promotion.**
 7. **Self-healing requires clear field ownership.**
 8. **Ignore rules must be narrow and evidence-based.**
 9. **Git revert is the preferred auditable rollback.**
 10. **Admission policy must protect the cluster even from an approved repository mistake.**
 11. **Argo Rollouts should use reliable runtime metrics, not weak model-quality signals.**
 12. **Secrets stay in secret-management systems; Git stores references.**
 13. **CRDs and cluster-wide controllers belong to a more privileged platform project.**
 14. **A Synced application can still be broken.**
 15. **The ability to merge a production PR and the ability to sync production should be separated.**
-

28. Interview Q&A

Q1. What is the main difference between Argo CD and a traditional deployment pipeline?

A traditional pipeline pushes changes to Kubernetes. Argo CD pulls desired state from Git and continuously reconciles the cluster. This gives built-in drift detection, self-healing and a Git-based audit trail.

Q2. Does Argo CD replace CI?

No. CI tests, evaluates, scans, builds, signs and publishes immutable artifacts. A controlled change records an approved artifact in the deployment repository. Argo CD reconciles that desired state to Kubernetes.

Q3. What is the difference between sync status and health status?

Sync status indicates whether live resources match Git. Health status indicates whether the deployed resources are operational. An application can be Synced but Degraded, or OutOfSync while its old live version remains Healthy.

Q4. When would you enable prune and selfHeal?

I normally enable both in development. In production I enable them only after resource ownership, deletion safeguards and repository controls are mature. I combine pruning with `allowEmpty: false`, `PruneLast=true` and confirmation for high-risk resources.

Q5. Why is Git revert preferred over Argo CD history rollback?

A Git revert restores Git as the authoritative desired state and records the recovery in the normal approval history. An Argo history rollback can be useful for emergency restoration, but Git must be corrected immediately or reconciliation may redeploy the bad state.

Q6. How would you represent a GenAI release in Git?

I would store an immutable application image digest plus model revision, prompt bundle hash, embedding model contract, vector-index schema and snapshot, dataset version, reranker version and evaluation-evidence reference. Promotion moves the complete compatible tuple between environments.

Q7. How do you prevent an incompatible vector index from reaching production?

I use PR schema validation and compatibility policies, then a PreSync Job checks the live index metadata against the release contract. The production release references the exact evaluated index snapshot, embedding model, dimensions and normalization settings.

Q8. How would you secure a multi-team Argo CD installation?

I would use SSO, group-based RBAC, restricted AppProjects, narrowly scoped repository and cluster credentials, protected Git branches, separate production sync permissions, admission policy, network isolation, audit logs and potentially separate Argo instances for strong trust boundaries.

Q9. When should ignoreDifferences be used?

Only when a known trusted controller legitimately owns a specific field and the ownership cannot be represented more cleanly. I first inspect `managedFields`, then ignore the smallest possible path on the specific resource. I avoid broad group- or kind-wide rules.

Q10. How do Argo CD and Argo Rollouts work together?

Argo CD reconciles the desired Rollout, Services and AnalysisTemplates from Git. The Argo Rollouts controller performs the canary or blue-green progression and evaluates runtime metrics. Argo CD defines the desired rollout strategy; Rollouts executes it.

Q11. What happens if Git becomes unavailable?

Running Kubernetes workloads continue serving. Argo CD cannot reliably fetch or render new desired state and will report repository or comparison errors. I restore repository access, hard-refresh the application, inspect the diff and only then resume synchronization.

Q12. How would you investigate an application that continuously becomes OutOfSync?

I would inspect the exact diff and Kubernetes managedFields, identify which controller changes the field, and determine whether Git, an autoscaler, operator or mutating webhook should own it. I would fix competing ownership or add a narrowly scoped ignore rule rather than disabling drift detection.

Final mental model

```
CI proves and publishes an immutable release
      ↓
Git records which release is approved where
      ↓
Argo CD continuously compares Git with EKS
      ↓
Sync applies the approved desired state
      ↓
Kubernetes and Rollouts manage workload progression
      ↓
Argo reports both configuration sync and runtime health
```

The most important interview statement is:

Argo CD does not decide what artifact is good enough for production. CI and the release-governance process establish that evidence. Argo CD ensures that the approved Git state is continuously and audibly reconciled into Kubernetes.